University of Ottawa

CSI 2110 - Data Structures and Algorithms

Laboratory 2

Linear Data Structures: Stacks, Queues, Deques, Lists, and Iterators

Professor	Mohamed Ali Ibrahim, PhD, PEng
Term	Spring/Summer 2026
Lab Statement Available	Tuesday, May 26, 2026
Lab Period	Tuesday, May 26, 2026 to Saturday, June 13, 2026
First Lab Session	Friday, May 29, 2026
Submission	Brightspace, by Saturday, June 13, 2026, 11:59 PM
Programming Language	Java

1. Laboratory Overview

The purpose of this laboratory is to help students implement and compare several linear Abstract Data Types (ADTs) used in data structures and algorithms. This lab is aligned with the topics covered from Chapter 6 and Chapter 7 of the course textbook and focuses on stacks, queues, deques, list ADTs, positional lists, iterators, and selected Java Collections Framework classes.

- Stacks and stack applications
- Queues, circular queues, and deques
- List ADT, array lists, and dynamic arrays
- Positional lists and iterators
- Java Collections Framework
- Complexity analysis and implementation trade-offs
- Java implementation, testing, and reporting

2. Topics Covered in Lab 2

Chapter Area	Topics Included in Lab 2
Chapter 6 - Stacks, Queues, and Deques	Stack ADT; stack applications; queue ADT; circular queue; deque ADT; array-based and linked implementations; practical use cases.
Chapter 7 - List and Iterator ADTs	List ADT; array lists; dynamic arrays; positional lists; iterators; iterable collections; comparison with Java Collections Framework classes.

3. Learning Objectives

- Implement stack, queue, circular queue, deque, list, and positional-list structures in Java.
- Use array-based, circular-array, linked, and dynamic-array representations appropriately.
- Apply stacks to solve a practical problem such as expression checking, undo operations, or reversal.
- Apply queues and deques to simulate scheduling, service lines, or task processing.
- Implement an iterator to traverse a custom data structure safely and clearly.
- Compare custom implementations with classes from the Java Collections Framework.
- Analyze the time complexity of the required operations using Big-O notation.
- Prepare a short technical report explaining design choices, testing results, and complexity analysis.

4. Problem Description

Students will design and implement a small Ticket Processing and Navigation System. The system models a simplified service desk where users submit tickets, tickets wait in different queues, recently processed tickets can be undone, and active tickets can be stored and traversed using list-based structures.

The lab is divided into required parts. Each part may be implemented in a separate class, but all components must be demonstrated through a complete Main.java test program. The goal is not only to make the code work, but also to explain the design decisions and the efficiency of each operation.

5. Required Data Model

Students must define a simple Ticket class similar to the following:

```
public class Ticket {
    private int id;
    private String requester;
    private String description;
    private int priority;

    // constructors, getters, setters, and toString()
}
```

Each ticket must have at least an integer ID, a requester name, a short description, and a priority value. Students may add other fields if they clearly explain them in the report.

6. Part A - Stack ADT and Stack Application

Create a generic stack interface and at least one implementation.

```
public interface Stack<E> {
    int size();
    boolean isEmpty();
    void push(E e);
    E top();
    E pop();
}
```

Requirements

- Implement ArrayStack<E> using an array.
- Implement LinkedStack<E> or clearly justify why only one implementation was selected.
- Use the stack to support an undo feature for recently processed tickets.
- Add one additional stack application, such as checking balanced parentheses in a string, reversing a sequence of tickets, or evaluating a simple postfix expression.

Expected behavior example:

```
process Ticket #101
process Ticket #102
undo last action  -> Ticket #102 returns to the appropriate waiting structure
```

7. Part B - Queue and Circular Queue

Create a generic queue interface and implement a circular-array queue for normal ticket processing.

```
public interface Queue<E> {
    int size();
    boolean isEmpty();
    void enqueue(E e);
    E first();
    E dequeue();
}
```

Requirements

- Implement CircularArrayQueue<E> using a fixed-size or dynamically resized circular array.
- The implementation must correctly handle wrap-around behavior.
- Demonstrate enqueue, dequeue, first, size, and isEmpty operations.
- Test edge cases such as an empty queue, a full array before resizing, and wrap-around after several enqueue/dequeue operations.

8. Part C - Deque ADT

Create and implement a generic deque structure for tickets that may be inserted or removed from both ends.

```
public interface Deque<E> {
    int size();
    boolean isEmpty();
    void addFirst(E e);
    void addLast(E e);
    E first();
    E last();
    E removeFirst();
    E removeLast();
}
```

Requirements

- Implement LinkedDeque<E> using a doubly linked structure.
- Use the deque to support urgent tickets being added to the front and normal tickets being added to the back.
- Demonstrate removing tickets from both ends.
- Explain when a deque is more appropriate than a queue or a stack.

9. Part D - List ADT, Array Lists, and Dynamic Arrays

Students must implement a simplified generic list ADT using a dynamic array.

```
public interface SimpleList<E> {
    int size();
    boolean isEmpty();
    E get(int index);
    E set(int index, E element);
    void add(int index, E element);
    E remove(int index);
}
```

Create a class:

```
public class DynamicArrayList<E> implements SimpleList<E>
```

Requirements

- Use an internal array to store elements.
- Resize the array when it becomes full.
- Shift elements correctly during add(index, element) and remove(index).
- Store completed tickets in this list and allow retrieval by index.
- Test invalid index handling. Students may throw an IndexOutOfBoundsException with a clear message.

10. Part E - Positional List and Iterators

Students must implement a small positional list to represent active tickets. The implementation may be based on a doubly linked list with header and trailer sentinels.

Required Operation	Description
first(), last()	Return the first or last position in the list.
before(p), after(p)	Return the position before or after position p.
addFirst(e), addLast(e)	Insert an element at the beginning or end of the list.
addBefore(p, e), addAfter(p, e)	Insert an element relative to an existing position.
set(p, e), remove(p)	Replace or remove the element stored at a position.
iterator()	Return an iterator that traverses the elements in order.

Iterator requirement: The iterator must be demonstrated in Main.java using a for-each style loop if the class implements Iterable<E>, or using explicit hasNext() and next() calls.

11. Part F - Java Collections Framework Comparison

Students must compare their custom implementations with equivalent Java Collections Framework classes.

Custom Structure	Suggested Java Class for Comparison
ArrayStack<E> or LinkedStack<E>	ArrayDeque<E>
CircularArrayQueue<E>	ArrayDeque<E> or LinkedList<E>
LinkedDeque<E>	ArrayDeque<E> or LinkedList<E>
DynamicArrayList<E>	ArrayList<E>
Positional list implementation	LinkedList<E> as an approximate comparison

In the report, students must explain at least two similarities and two differences between their custom implementations and the selected Java library classes. The comparison should focus on behavior, efficiency, safety, and ease of use.

12. Part G - Complexity Analysis

Students must complete a written complexity analysis section in the report. The analysis must include Big-O notation and short justifications.

Required Complexity Table

Operation	ArrayStack	LinkedStack	CircularArrayQueue	LinkedDeque	DynamicArrayList	PositionalList
push / addFirst						
pop / removeFirst						
enqueue / addLast						
dequeue						
get(index)						
set(index, element)						
add(index, element)						
remove(index)						
iterator traversal						

Required Written Questions

- Explain why a circular array can avoid unnecessary shifting in queue operations.
- Explain the difference between using a stack and using a queue in a real application.
- Explain why add(index, element) in a dynamic array may require shifting elements.
- Explain why a doubly linked deque can perform insertion and deletion at both ends efficiently.
- Explain the purpose of an iterator and why it is useful for hiding implementation details.

- Compare the memory trade-offs of array-based and linked implementations.

13. Testing Requirements

Students must create a Main.java file that demonstrates the correctness of all required implementations and exercises. The test program must include:

- Creating at least 8 ticket objects.
- Testing stack push, top, pop, and the undo feature.
- Testing one additional stack application.
- Testing circular queue enqueue, dequeue, first, wrap-around behavior, and empty queue behavior.
- Testing deque insertion and removal at both ends.
- Testing DynamicArrayList get, set, add, remove, resizing, and invalid index handling.
- Testing positional-list insertion, removal, replacement, traversal, and iterator behavior.
- Testing comparison examples using Java Collections Framework classes.
- Testing edge cases: empty structure, one-element structure, removing from an empty structure, and invalid position or index.

14. Deliverables

Each group must submit a ZIP file on Brightspace containing:

- Ticket.java
- Stack.java, ArrayStack.java, and optionally LinkedStack.java
- Queue.java and CircularArrayQueue.java
- Deque.java and LinkedDeque.java
- SimpleList.java and DynamicArrayList.java
- Position.java, PositionalList.java, and a linked positional-list implementation
- Main.java
- A short lab report in PDF format

The report should include:

- Team members' names and student numbers
- Description of the implementation and design choices
- Screenshots or copied output from test runs
- Complexity analysis table
- Answers to the required written questions
- Short discussion comparing stack, queue, deque, list, positional list, and Java Collections Framework classes
- Problems encountered and how they were solved

15. Laboratory Demonstration Requirement

The laboratory demonstration is a mandatory component of Lab 2. Each team must demonstrate its implementation to the Teaching Assistant (TA).

- Each team must complete a demonstration with the TA for the lab report to be considered for grading.
- All team members must be present during the demonstration.
- Each member may be asked to explain the implementation, tests, iterator behavior, and complexity analysis.
- The demonstration should clearly show that the submitted code runs correctly and matches the requirements of this lab.
- If a team does not complete the laboratory demonstration with the TA, the maximum mark the team can receive for the lab is 50%. This maximum would only apply to an exceptionally strong and complete written report. In general, without the required demonstration, the final lab grade is expected to be significantly lower than 50%.

16. Grading Rubric

Total: 100 points

Component	Points
Stack ADT implementation and stack application	15
Queue and circular-array queue implementation	15
Deque implementation and use case	12
DynamicArrayList and List ADT implementation	15
Positional list and iterator implementation	15
Java Collections Framework comparison	8
Testing, edge cases, and demonstration quality	10
Complexity analysis and written justification questions	7
Code quality, comments, organization, and report quality	3
Total	100

17. Important Notes

- Late submissions are not accepted unless an official extension has been granted by the professor.
- All team members must understand the submitted code.
- Students may be asked to explain their implementation during the lab or tutorial.
- Code must be written in Java.
- Plagiarism or copied code will receive a grade of zero and may be reported according to university academic integrity policies.
- Students may use the Java Collections Framework only in Part F for comparison, unless explicitly stated otherwise. The required custom structures must be implemented by the students.

18. Timeline

Date	Work
Tuesday, May 26	Lab statement becomes available. Read the full statement, review the ADTs, and create the Java project structure.
May 27-28	Review Ch. 6 and Ch. 7. Begin Ticket.java, Stack.java, ArrayStack.java, and the stack application.
Friday, May 29	First lab session. Confirm requirements with the TA, ask questions, and continue testing the stack application.
May 30-31	Implement Queue.java and CircularArrayQueue.java; test wrap-around and empty-queue cases.
June 1-3	Implement Deque.java and LinkedDeque.java; test both-end operations.
June 4-6	Implement SimpleList.java and DynamicArrayList.java; test resizing and index operations.
June 7-9	Implement the positional list and iterator; test traversal and position operations.
June 10	Complete the Java Collections Framework comparison.
June 11	Complete the complexity analysis and written justification questions.
June 12	Complete final testing, prepare output/screenshots, and write the report.
Saturday, June 13	Submit the ZIP file and laboratory report on Brightspace by 11:59 PM.